Propagator Networks

A Comprehensive Survey of Theory, Practice, and Applications

The Prologos Project - from Zachary Larson

2026-03-05

Contents

1 Introduction **3**

2 Origins and History **4**
2.1 Constraint Propagation in Circuit Analysis (1977) 4
2.2 Steele's Constraint Language (1980) 4
2.3 Truth Maintenance Systems (1979–1986) 4
2.4 Waltz Filtering and Arc Consistency (1975–1977) 5
2.5 Dataflow Programming (1974) 5
2.6 Radul's PhD Thesis (2009) 6
2.7 "The Art of the Propagator" (2009) 6
2.8 The Revised Report (2010) 6
2.9 Software Design for Flexibility (2021) 6

3 Core Concepts **7**
3.1 Cells 7
3.2 Propagators 7
3.3 The Merge Operation 8
3.4 Partial Information 9
3.5 Monotonicity 9
3.6 Quiescence and Fixed Points 10
3.7 Multidirectional Computation 10
3.8 Supported Values and Dependencies 11
3.9 Truth Maintenance Integration 11

4 Lattice-Theoretic Foundations **12**
4.1 Lattices and Partial Orders 12
4.2 Merge as Lattice Join 12
4.3 Monotone Functions and Convergence 12
4.4 The CALM Theorem 13
4.5 Widening and Narrowing 13

5 Relationship to Other Computational Paradigms **14**
5.1 Constraint Programming 14
5.2 Datalog and Fixed-Point Logic 14
5.3 Reactive Programming and Spreadsheets 15
5.4 SAT/SMT Solvers 15
5.5 The Actor Model 16
5.6 CRDTs and Distributed Systems 17

6 Applications **18**
6.1 Type Inference 18
6.2 Constraint Solving 18
6.3 Abstract Interpretation 19
6.4 Protocol Verification via Session Types 19
6.5 Incremental Compilation 20
6.6 Diagnostic Reasoning 20

7 Implementations **20**
7.1 The Reference Implementation (Scheme) 20
7.2 Haskell Libraries 21
7.3 Clojure Libraries 21
7.4 Constraint Programming Systems 21

8 A Worked Example: The Prologos Propagator Architecture **21**
8.1 Design Principles 22
8.2 Core Data Structures 22
8.3 The Three Lattice Domains 23
8.3.1 Type Lattice 23
8.3.2 Multiplicity Lattice 23
8.3.3 Session Lattice 23
8.4 Cross-Domain Propagation via Galois Connections 24
8.5 Scheduling Strategies 24
8.5.1 Gauss-Seidel (Sequential) 24
8.5.2 BSP (Parallel) 25

8.6 Structural Decomposition 25
8.7 Example: Type Inference as Propagation 25
8.8 Propagators as Universal Substrate 26

9 Future Directions **26**
9.1 Theorem Proving via Abstract CDCL 26
9.2 User-Defined Constraint Domains 26
9.3 Distributed Propagation 26
9.4 Incremental and Live Programming 27
9.5 Provenance and Explainability 27

10 References **27**

1 Introduction

A **propagator network** is a model of computation in which autonomous, stateless computational agents — called **propagators** — communicate through shared stateful storage elements — called **cells** — that accumulate partial information about values. Unlike traditional models where programs execute as sequential instructions over mutable memory, propagator networks describe computation as the gradual, multidirectional accumulation of knowledge, driven by local rules, until no further deductions can be made.

The key insight, due to Alexey Radul [1], is that **a cell should not be seen as storing a value, but as accumulating information about a value**. This reframing transforms cells from simple memory locations into lattice-structured accumulators of partial information, and elevates the computational model from "compute a result" to "refine what we know until we know enough."

Propagator networks unify ideas from constraint propagation, truth maintenance, dataflow programming, and lattice-theoretic fixed-point computation into a single, composable framework. They have applications in type inference, constraint solving, abstract interpretation, incremental compilation, reactive programming, and protocol verification. Despite this breadth, the core model is remarkably simple: cells hold lattice elements; propagators are monotone functions between them; and the network runs until quiescence (a fixed point).

[1] Radul, A. (2009). "Propagation Networks: A Flexible and Expressive Substrate for Computation." PhD thesis, MIT.

There is, as of this writing, no Wikipedia article on propagator networks. This document aims to fill that gap: a self-contained survey accessible to a reader with general programming background, progressing from foundations through theory to applications and the state of the art.

2 Origins and History

2.1 Constraint Propagation in Circuit Analysis (1977)

The propagator model traces its lineage to Gerald Jay Sussman and Richard Stallman's 1977 work on computer-aided circuit analysis

2.2 Steele's Constraint Language (1980)

Guy Lewis Steele Jr., also working under Sussman at MIT, developed the theoretical foundations of constraint-based programming in his 1980 PhD thesis [2]. His key contribution was the **locality principle**: each constraint operates using only locally available information and places newly derived values on locally attached wires. No constraint needs global knowledge of the network; all computation is local.

This principle became a cornerstone of the propagator model. It is what makes propagator networks inherently parallelizable: if every computational element operates independently using only local information, there is no need for global coordination.

2.3 Truth Maintenance Systems (1979–1986)

Sussman's circuit analysis work spawned a line of research into **truth maintenance** — systems that track *why* facts are believed, not just *what* is believed.

Jon Doyle's TMS (1979) [3] introduced the justification-based truth maintenance system. Each belief in the system is annotated with its justification — the set of prior beliefs and inference rules that support it. When a supporting belief is retracted, all conclusions that depended on it are automatically invalidated. This enabled non-monotonic reasoning: the system can change its mind as assumptions change.

[2] Steele, G.L. (1980). "The Definition and Implementation of a Computer Programming Language Based on Constraints." PhD thesis, MIT. AI Laboratory TR-595.

[3] Doyle, J. (1979). "A Truth Maintenance System." *Artificial Intelligence*, 12:231–272.

Johan de Kleer's ATMS (1986) [4] advanced the idea significantly with the assumption-based truth maintenance system. Rather than maintaining one "current" set of beliefs (as the TMS does), the ATMS simultaneously tracks *all possible* assumption sets — each a consistent "worldview." With n assumptions, there are potentially 2^n environments. Since all environments are maintained in parallel, switching between worldviews is free (no retraction or re-derivation needed).

The ATMS proved essential for the propagator model's handling of hypothetical reasoning and search, as we will see.

2.4 Waltz Filtering and Arc Consistency (1975–1977)

Independently from the Sussman lineage, David Waltz's 1975 algorithm for interpreting line drawings of 3D scenes [5] demonstrated that local constraint propagation could dramatically reduce search spaces. Alan Mackworth formalized this as **arc consistency** in 1977

2.5 Dataflow Programming (1974)

Jack Dennis formalized dataflow schemas in 1974 [6]: programs as directed graphs where operations execute upon data availability, rather than following a sequential program counter. Gilles Kahn independently introduced Kahn Process Networks [7] — sequential processes communicating through FIFO queues — and proved that such networks are deterministic: the output sequence does not depend on process execution speed.

Propagator networks generalize dataflow in two ways: (1) information flows *multidirectionally* (not just producer-to-consumer), and (2) cells accumulate *partial information* (not complete values). A standard dataflow network is a special case of a propagator network where all propagators are unidirectional and all cells hold complete values.

[4] de Kleer, J. (1986). "An Assumption-Based TMS." *Artificial Intelligence*, 28(2):127–162.

[5] Waltz, D. (1975). "Understanding Line Drawings of Scenes with Shadows." In *The Psychology of Computer Vision*, McGraw-Hill.

[6] Dennis, J.B. (1974). "First Version of a Data Flow Procedure Language." *Lecture Notes in Computer Science*, vol 19, Springer.

[7] Kahn, G. (1974). "The Semantics of a Simple Language for Parallel Programming." *IFIP Congress*, pp. 471–475.

2.6 Radul's PhD Thesis (2009)

Alexey Radul's 2009 PhD thesis at MIT, "Propagation Networks: A Flexible and Expressive Substrate for Computation" [1], supervised by Sussman, is the definitive formulation of the propagator model as a general-purpose computational paradigm. The thesis synthesized three decades of ideas — constraint propagation, truth maintenance, lattice theory, dataflow — into a unified model with a precise mathematical foundation.

Radul argued that conventional programming systems are insufficiently expressive because they are built on the image of a single computer with global effects on a large memory. The propagator model offers an alternative: autonomous machines communicating through shared cells, where computation is the progressive accumulation of partial information about values.

The thesis demonstrated that an astonishing range of computational paradigms embed naturally into propagator networks: expression evaluation, constraint satisfaction, truth maintenance, functional reactive programming, rule-based systems, and type inference are all instances of the same underlying pattern.

2.7 "The Art of the Propagator" (2009)

Radul and Sussman published a companion paper, "The Art of the Propagator" [8], presenting the model more concisely. The paper emphasizes that the model "makes it easy to smoothly combine expression-oriented and constraint-based programming, and easily accommodates implicit incremental distributed search in ordinary programs."

2.8 The Revised Report (2010)

Radul and Sussman published the "Revised Report on the Propagator Model"

2.9 Software Design for Flexibility (2021)

Chris Hanson and Sussman's book *Software Design for Flexibility*

[8] Radul, A. & Sussman, G.J. (2009). "The Art of the Propagator." MIT-CSAIL-TR-2009-002.

3 Core Concepts

3.1 Cells

A **cell** is the stateful storage element of a propagator network. Unlike a variable in traditional programming (which holds a single definitive value and can be overwritten), a cell *accumulates information about a value.* A cell supports three operations:

1. **Add content**: New information is *merged* with the cell's existing information (not overwritten).
2. **Read content**: Retrieve the current state of accumulated knowledge.
3. **Register propagators**: Propagators subscribe to the cell and are notified when its content changes.

The information in a cell ranges from "I know absolutely nothing" (the special value `nothing`, or lattice bottom $\perp$) to "I know everything there is to know, and the answer is 42" to the anomalous case "I know there is a contradiction!" (lattice top $\top$).

Critically, information in a cell *never decreases.* A cell can move from "I don't know" to "it's an integer" to "it's a positive integer" to "it's 42," but it can never move backward from "it's 42" to "I don't know." This monotonicity is not merely a convention — it is the mathematical foundation ensuring convergence and determinism.

3.2 Propagators

A **propagator** is the computational element of the network. Propagators are:

- **Autonomous**: They operate independently, with no central controller directing their execution.
- **Stateless**: They maintain no internal state. All persistent state resides in cells.
- **Asynchronous**: They can fire in any order; the final result is the same (confluence).

A propagator monitors its input cells and, when any input changes, computes new information and adds it to its output cells. The computation

is always *additive* — a propagator can only *contribute* information, never *remove* it.

In practice, a propagator is a function from the current network state to a (possibly modified) network state: it reads from some cells, computes, and writes to others via the merge operation. If its contribution adds no new information (i.e., the merge result is identical to the cell's existing content), the network state is unchanged and no further propagation is triggered.

3.3 The Merge Operation

The merge operation is the heart of the propagator model. When a propagator writes information b to a cell already containing information a, the cell's new content is $\text{merge}(a, b)$ — the combination of both pieces of knowledge.

Merge must satisfy three algebraic laws:

- **Commutativity**: $\text{merge}(a, b) = \text{merge}(b, a)$ — the order in which information arrives does not matter.
- **Associativity**: $\text{merge}(\text{merge}(a, b), c) = \text{merge}(a, \text{merge}(b, c))$ — grouping does not matter.
- **Idempotency**: $\text{merge}(a, a) = a$ — re-contributing the same information has no effect.

These three laws make merge a **join-semilattice** operation (see 4). The commutativity and associativity ensure that the final cell state is independent of the order in which propagators fire. The idempotency ensures that redundant contributions are harmless. Together, these properties guarantee **confluence**: the result of running a propagator network depends only on the initial cell contents and the set of propagators, not on the order or timing of execution.

Key behaviors of merge:

- $\text{merge}(\bot, x) = x$ — merging anything with "nothing" yields that thing (bottom is the identity).
- $\text{merge}(x, x) = x$ — consistent redundant information is harmless.
- $\text{merge}(\top, x) = \top$ — once a contradiction is reached, it cannot be undone (top is absorbing).
- $\text{merge}(a, b) = \top$ when a and b are incompatible — contradictory information produces a contradiction.

3.4 Partial Information

The propagator model supports a rich hierarchy of partial information types:

- **Nothing ($\bot$)**: Complete ignorance about the value.
- **Raw values**: Fully determined information (e.g., the integer 42).
- **Intervals**: Bounded ranges (e.g., $[3.0, 5.0]$), representing the knowledge that a value lies somewhere within bounds.
- **Finite domains**: Sets of possible values (e.g., $\{1, 2, 3, 4\}$).
- **Supported values**: Values paired with the set of assumptions that justify them (see 3.8).
- **TMS entries**: Sets of supported values, representing multiple contingent beliefs that coexist.

Each partial information type defines three generic operations: `equivalent?` (test whether two structures represent the same information), `merge` (combine two structures), and `contradictory?` (test whether a structure represents an impossible state).

The power of the propagator model comes from the ability to define *new* partial information types. Any data type that forms a join-semilattice with a bottom element and a contradiction test can be used as cell content. This extensibility is what allows propagator networks to operate over types, multiplicities, session protocols, intervals, finite domains, or any other structured form of partial knowledge.

3.5 Monotonicity

All information flow in a propagator network must be **monotonic**: cells can only gain information, never lose it. In lattice-theoretic terms, the content of a cell can only move *upward* in the information ordering (toward more-determined states, further from $\bot$, closer to $\top$).

Monotonicity is the property that makes everything else work:

- **Confluence**: Because merge is commutative and associative, and propagators can only increase cell values, the final result is independent of execution order.

- **Termination**: In a finite lattice, information can only increase a finite number of times before reaching a fixed point. (Infinite lattices require *widening* — see 4.5.)
- **Compositionality**: Networks can be freely combined without worrying about interference. Each sub-network's monotonic contributions compose with others.

3.6 Quiescence and Fixed Points

A propagator network reaches **quiescence** when no propagator has new information to contribute — the system has reached a **fixed point**. At quiescence, every cell contains the **least fixed point** of the system of equations defined by the propagators: the smallest (least-informative) cell contents consistent with all propagators being satisfied.

The existence and uniqueness of this least fixed point is guaranteed by the **Knaster-Tarski fixed-point theorem**: every monotone function on a complete lattice has a least fixed point, and it can be computed by iterating from $\bot$:

$$\mathrm{lfp}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\bot)$$

This is precisely what running a propagator network to quiescence computes: starting from cells containing $\bot$ (nothing), repeatedly firing propagators until nothing changes.

3.7 Multidirectional Computation

Unlike traditional functions (which map inputs to outputs in one direction), propagators support **multidirectional computation**. A constraint like $x + y = z$ is expressed not as a function from (x, y) to z, but as a set of propagators:

- One computes z from x and y
- One computes x from z and y
- One computes y from z and x

Given any two of the three values, the third is determined. Given all three, consistency is verified. This arises naturally from the merge semantics: each propagator contributes whatever it can, and merge reconciles all contributions.

The classic example (from Radul and Sussman) is temperature conversion. The relationship $C = (F - 32) \times 5/9$ is expressed once, and the network automatically solves for whichever variable is unknown — forward (F to C) or backward (C to F).

3.8 Supported Values and Dependencies

A **supported value** is a pair (v, P) of a datum v and a set of **premises** P — the assumptions on which v depends. This enables the system to track not just *what* it believes, but *why*.

When a propagator derives new information, the result is tagged with the union of its inputs' premise sets. If cell A holds $(3, \{p_1\})$ and cell B holds $(5, \{p_2\})$, an addition propagator writes $(8, \{p_1, p_2\})$ to cell C — "the sum is 8, given that we believe both p_1 and p_2."

When a contradiction is discovered, the system can identify *which premises* contributed. The conflicting premise set is recorded as a **nogood** — a set of assumptions that cannot all be simultaneously true. This enables **dependency-directed backtracking**: the system retracts the minimal set of assumptions needed to resolve the conflict, rather than blindly reverting the most recent choice.

3.9 Truth Maintenance Integration

Propagator networks integrate with **truth maintenance systems** to enable hypothetical reasoning. A TMS-backed cell contains a set of supported values, each contingent on different assumptions. When assumptions are activated or deactivated, the TMS automatically updates which values are "in" (currently believed) and which are "out."

The ATMS (Assumption-Based Truth Maintenance System) extends this to maintain *all possible* worldviews simultaneously. Rather than committing to one set of assumptions, the ATMS tracks every consistent combination. This enables:

- **Hypothetical reasoning**: "What if we assume p_1? What follows?" — without committing to p_1.
- **Dependency-directed backtracking**: When a contradiction is found, trace back to the responsible premises and record the nogood.
- **Non-monotonic reasoning**: While the underlying information flow is monotonic (lattice joins), the TMS layer enables *belief revision* by changing which premises are active.

4 Lattice-Theoretic Foundations

4.1 Lattices and Partial Orders

A **partial order** is a set S with a binary relation $\leq$ that is reflexive ($a \leq a$), antisymmetric ($a \leq b$ and $b \leq a$ implies $a = b$), and transitive ($a \leq b$ and $b \leq c$ implies $a \leq c$). The ordering represents "has at least as much information as."

A **join-semilattice** $(L, \leq, \bot, \sqcup)$ is a partial order with:

- A **bottom element** $\bot$ (least informative — "nothing")
- A **join operation** $\sqcup$ (least upper bound — "combine information")

The join is commutative, associative, and idempotent. Join is precisely the merge operation of propagator cells.

A **complete lattice** additionally has a **top element** $\top$ (maximally informative, representing contradiction) and a **meet** operation $\sqcap$ (greatest lower bound). In propagator networks, reaching $\top$ signals that incompatible information has been merged.

4.2 Merge as Lattice Join

The merge operation implements the lattice join:

$$\text{merge}(a, b) = a \sqcup b$$

All algebraic properties of merge follow from the lattice axioms:

Property	Equation	Consequence
Commutativity	$a \sqcup b = b \sqcup a$	Order-independence
Associativity	$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$	Grouping-independence
Idempotency	$a \sqcup a = a$	Redundancy-tolerance
Identity	$a \sqcup \bot = a$	Adding nothing changes nothing
Annihilation	$a \sqcup \top = \top$	Contradiction is irreversible

4.3 Monotone Functions and Convergence

A function $f : L \to L$ is **monotone** if $a \leq b$ implies $f(a) \leq f(b)$ — more information in, at least as much information out. Propagators must be monotone functions.

The **Knaster-Tarski fixed-point theorem** guarantees that every monotone function on a complete lattice has a least fixed point. Combined with Kleene's iteration theorem, this means:

1. A fixed point exists.
2. It can be computed by iterated application from $\perp$.
3. For finite lattices, iteration terminates in finitely many steps.
4. The result is independent of the iteration order.

Property (4) — *confluence* — is the deepest guarantee of the propagator model: no matter how propagators are scheduled, the network converges to the same result.

4.4 The CALM Theorem

The CALM theorem (Consistency As Logical Monotonicity) [9], formulated by Hellerstein and Alvaro, establishes that monotonic programs (those expressible as monotone functions on lattices) can be executed in a distributed, coordination-free manner while still producing consistent results. Non-monotonic operations (like negation or aggregation) require coordination (synchronization barriers).

Propagator networks are inherently monotonic, so the CALM theorem guarantees that they can be distributed across multiple processors or machines without coordination, and the result will be the same as single-machine execution. This makes propagator networks a natural fit for parallel and distributed computation.

4.5 Widening and Narrowing

For lattices of infinite height (e.g., the lattice of integer intervals $[lo, hi]$), Kleene iteration may not terminate — a cell could ascend through infinitely many refinements. **Widening** (due to Cousot & Cousot

[9] Hellerstein, J.M. & Alvaro, P. (2019). "Keeping CALM: When Distributed Consistency Is Easy." *Communications of the ACM*, 63(9).

5 Relationship to Other Computational Paradigms

5.1 Constraint Programming

Propagator networks subsume the constraint propagation phase of constraint programming (CP) systems. In CP, variables have domains (typically finite sets), and constraints are propagators that narrow domains by removing inconsistent values. Modern production CP solvers like Gecode [10] are explicitly built on propagator architectures.

The relationship is direct:

Constraint Programming	Propagator Networks
Variable domain	Cell content (lattice value)
Constraint propagator	Propagator (fire function)
Arc consistency (AC-3, etc.)	Run to quiescence
Backtracking search	ATMS worldview branching
Learned nogood	Recorded assumption conflict

Propagator networks generalize CP in that cells can hold *any* lattice-structured partial information, not just finite domains. An interval, a type, a session protocol, or a user-defined lattice are all valid cell contents.

5.2 Datalog and Fixed-Point Logic

Datalog — the database query language based on Horn clauses evaluated to a fixed point — is a direct instance of propagator-style computation on a specific lattice: sets of tuples ordered by subset inclusion.

- Each Datalog relation is a cell holding a set of tuples.
- Each Datalog rule is a propagator: when input relations change, derive new tuples for the head relation.
- Naive evaluation (iterate all rules until nothing changes) = run to quiescence.
- Semi-naive evaluation (only propagate *new* tuples) = an optimization of the propagator scheduling strategy.

[10] Schulte, C. & Stuckey, P.J. (2008). "Efficient Constraint Propagation Engines." *ACM Transactions on Programming Languages and Systems*, 31(1).

Flix [11], a research language by Madsen et al. (PLDI 2016), extends Datalog by replacing set inclusion with arbitrary user-defined lattices. Flix's model is essentially Datalog-as-propagator-network: lattice-valued relations, monotone rules, iterative fixed-point evaluation. Datafun [12] (Arntzenius & Krishnaswami, ICFP 2016) takes this further, embedding monotone fixed-point computation into a functional type system.

5.3 Reactive Programming and Spreadsheets

Spreadsheets are the most widely deployed example of propagator-like computation. When a cell changes, dependent cells recompute automatically — unidirectional propagation through a dependency graph.

Reactive programming generalizes this with observable streams and automatic change propagation. Functional reactive programming (FRP) adds compositional abstractions.

Propagator networks differ from reactive/FRP systems in three key ways:

1. **Multidirectionality**: Reactive systems propagate change in one direction (from sources to dependents). Propagators support multidirectional constraints.

2. **Partial information**: FRP cells hold complete values that are overwritten on change. Propagator cells accumulate partial information via merge.

3. **Merge semantics**: FRP cells are "last writer wins." Propagator cells are "all writers contribute" via lattice join.

Radul's thesis demonstrates that FRP embeds naturally within the propagator framework: a standard reactive system is a propagator network with unidirectional propagators and complete-value cells. David Thompson's work [13] shows the practical value of this embedding for building functional reactive user interfaces with propagators.

5.4 SAT/SMT Solvers

The relationship between propagators and SAT/SMT solvers is structural. Modern SAT solvers use CDCL (Conflict-Driven Clause Learning):

[11] Madsen, M., Yee, M.-H., & Lhotak, O. (2016). "From Datalog to Flix: A Declarative Language for Fixed Points on Lattices." *PLDI 2016*.

[12] Arntzenius, M. & Krishnaswami, N.R. (2016). "Datafun: A Functional Datalog." *ICFP 2016*.

[13] Thompson, D. (2014). "Functional Reactive User Interfaces with Propagators."

1. **Decision**: Choose an unassigned variable and assign it.
2. **Unit propagation**: Deduce forced assignments from clauses.
3. **Conflict analysis**: When a contradiction is found, learn a new clause (nogood).
4. **Non-chronological backjumping**: Jump back to the most relevant decision level.

Each of these maps directly to propagator concepts:

CDCL	Propagator Network
Decision literal	ATMS assumption
Unit propagation	Run to quiescence
Conflict clause	Nogood set
Non-chronological backjump	Dependency-directed backtracking
Theory solver	Domain-specific propagators

SMT (Satisfiability Modulo Theories) extends SAT with "theory propagators" that participate in propagation and conflict analysis — the same architectural pattern as domain-specific propagators in a propagator network.

D'Silva et al. [14] showed that CDCL can be lifted from the Boolean lattice to arbitrary lattice-based abstract domains ("Abstract CDCL"). This directly connects SAT/SMT solving to the lattice-theoretic core of propagator networks.

5.5 The Actor Model

Carl Hewitt's Actor Model [15] describes computation as autonomous agents (actors) communicating through asynchronous messages. Propagator networks resemble actor systems architecturally — both feature autonomous computational elements without central control — but differ crucially:

[14] D'Silva, V. et al. (2013). "Abstract Conflict Driven Learning." *SAS 2013*.

[15] Hewitt, C. (1973). "A Universal Modular ACTOR Formalism for Artificial Intelligence."

Dimension	Actors	Propagators
Communication	Point-to-point messages	Shared cells with merge
State	Per-actor mutable state	Shared cells accumulate info
Determinism	Non-deterministic	Deterministic (confluence)
Convergence	No guarantee	Guaranteed (lattice theory)
Fault model	Supervision, let-it-crash	Contradiction + ATMS nogood

The lattice constraint on cell values is what buys determinism: unlike actors, where message ordering can affect the outcome, the commutativity of merge ensures that propagator execution order is irrelevant.

Kuper and Newton's LVars [16] (see) formalize this connection: LVars are shared mutable variables with monotonic writes and threshold reads, guaranteeing deterministic parallelism — precisely the semantics of propagator cells.

5.6 CRDTs and Distributed Systems

Conflict-Free Replicated Data Types (CRDTs) [17] share the same mathematical substrate as propagator cells. State-based CRDTs (CvRDTs) require:

- A join-semilattice structure on state
- A merge function that is commutative, associative, and idempotent
- Monotonic state updates

These are *exactly* the requirements for propagator cell contents. Both CRDTs and propagators solve the problem of combining information from multiple autonomous sources in a consistent, order-independent way. CRDTs solve it for distributed replicas across a network; propagators solve it for computational elements within a program.

The CALM theorem [9] makes this connection precise: monotonic computations (like propagator networks and CRDT merges) can be distributed without coordination. Edward Kmett's influential talks on propagators [18] explicitly identify this unification: "propagators, CRDTs, Datalog, SAT solving, and FRP all share the same lattice-fixpoint structure."

[16] Kuper, L. & Newton, R.R. (2013). "LVars: Lattice-based Data Structures for Deterministic Parallelism." *FHPC 2013*.

[17] Shapiro, M. et al. (2011). "Conflict-Free Replicated Data Types." *SSS 2011*.

[18] Kmett, E. (2016–2019). "Propagators." Talks at FnConf, YOW!, and others.

6 Applications

6.1 Type Inference

Type inference — the problem of determining the types of expressions in a program without explicit annotations — maps directly onto propagator networks. Radul's thesis includes a chapter titled "Type Inference Looks Like Propagation Too."

The mapping:

- Each type variable (metavariable) is a *cell*, initially containing $\bot$ ("I don't yet know what type this is").
- Each typing rule (function application, variable binding, etc.) is a *propagator* that constrains related type cells.
- Unification is *merge*: when two type cells are constrained to be equal, their contents are merged (unified). Compatible types merge to a more-specific type; incompatible types merge to $\top$ (type error).
- Running to quiescence computes the principal type (the most general type consistent with all constraints).

Bidirectional type checking — splitting inference into synthesis (bottom-up, from term to type) and checking (top-down, from expected type to term) — is a natural instance of multidirectional propagation. Recent work by Leijen [19] on "omnidirectional type inference" makes this explicit: typing constraints suspend when information is insufficient and resume when other constraints supply it — precisely propagator semantics.

6.2 Constraint Solving

Propagator networks are the native architecture of constraint solving. Variables are cells, constraints are propagators, and solving is running to quiescence. The advantages over traditional constraint solvers:

- **Mixed-domain solving**: Finite domain constraints, interval constraints, type constraints, and user-defined constraints can coexist in a single network, connected by cross-domain propagators.

[19] Leijen, D. (2025). "Omnidirectional Type Inference for ML."

- **Persistent backtracking**: If the network uses persistent (immutable) data structures, backtracking is $O(1)$ — just keep the old network reference. Traditional CP systems pay $O(\text{trail length})$ per backtrack.
- **Extensibility**: Adding a new constraint domain is defining a new lattice and wiring propagators. In traditional systems (Gecode, MiniZinc), adding a domain requires low-level implementation.

6.3 Abstract Interpretation

Abstract interpretation [20] — computing sound approximations of program behavior over abstract domains — is propagator networks operating over abstract lattices:

- Each *program point* is a cell holding an abstract value (e.g., sign, interval, pointer alias set).
- Each *program statement* is a propagator computing abstract transfer functions.
- Running to quiescence computes the analysis fixed point.
- Widening handles infinite-height abstract domains.

The connection to propagators is made precise by **Galois connections**: pairs of monotone functions $(\alpha : C \to A, \gamma : A \to C)$ connecting concrete and abstract domains. Cross-domain propagators use Galois connections to flow information between multiple abstract domains simultaneously — the "reduced product" of multiple analyses becomes automatic when all domains share a propagator network.

6.4 Protocol Verification via Session Types

Session types — type-theoretic specifications of communication protocols — can be verified using propagator networks. Each channel endpoint is a cell in a session type lattice; each communication operation (send, receive, select, offer) is a propagator that constrains the channel's protocol.

[20] Cousot, P. & Cousot, R. (1977). "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL 1977*.

Caires and Pfenning [21] established a Curry-Howard correspondence between session types and intuitionistic linear logic. Propagator-based session type checking exploits this: the duality constraint (client and server must follow dual protocols) is a bidirectional propagator, and protocol violations are contradictions (lattice top) with dependency chains explaining *which* communication step violated *which* protocol clause.

6.5 Incremental Compilation

An incremental compiler can be structured as a propagator network:

- Source files are cells (content = parsed AST).
- Compilation phases (parse → elaborate → type-check → codegen) are propagator chains.
- A file change updates a cell; running to quiescence recomputes only affected downstream cells.
- Persistent networks give $O(1)$ access to the previous version for delta computation.

The multidirectionality of propagators adds a novel capability: a type signature change can propagate *backward* to callers, enabling "what-if" analysis ("what happens if I change this type?") without recompiling. This connects to Acar's work on self-adjusting computation

6.6 Diagnostic Reasoning

De Kleer and Williams's General Diagnostic Engine (GDE, 1987)

7 Implementations

7.1 The Reference Implementation (Scheme)

The canonical implementation is Radul and Sussman's Scheme-Propagators

[21] Caires, L. & Pfenning, F. (2010). "Session Types as Intuitionistic Linear Propositions." *CONCUR 2010*.

7.2 Haskell Libraries

Edward Kmett's `propagators` [22]: An exploration of propagator design in Haskell. The primary innovation beyond the published Scheme work is the use of **observable sharing** to let users write in a direct programming style that is automatically transformed to and from propagator form.

Holmes [23]: A reference library for constraint-solving with propagators and CDCL. Holmes exposes two strategies: `satisfying` (returns first valid configuration) and `whenever` (returns all valid configurations). Its README demonstrates a complete Sudoku solver.

7.3 Clojure Libraries

propaganda [24]: A Clojure propagator library implementing the model from "The Art of the Propagator." Offers both an STM-based and an immutable implementation strategy.

7.4 Constraint Programming Systems

Gecode [10]: A production-quality, highly efficient constraint solver explicitly built on a propagator architecture. Gecode demonstrates that the propagator architecture is viable for industrial-strength constraint solving. Its propagation kernel is domain-independent, and propagators are first-class objects that can be dynamically added to a constraint model.

Most modern CP systems (SICStus Prolog, ECLiPSe, SWI-Prolog's CLP libraries) use propagator-based architectures internally, where each constraint is a propagator that narrows variable domains.

8 A Worked Example: The Prologos Propagator Architecture

To illustrate how propagator networks function as practical infrastructure, we describe the propagator architecture of Prologos — a programming language that uses propagator networks as the unified substrate for type inference, multiplicity tracking (QTT), and session type verification.

[22] Kmett, E. Haskell `propagators` library. `https://github.com/ekmett/propagators`

[23] Holmes (i-am-tom). Haskell constraint-solving with propagators and CDCL. `https://github.com/i-am-tom/holmes`

[24] Nilsson, T.K. `propaganda` — Clojure propagator library. `https://github.com/tgk/propaganda`

8.1 Design Principles

Prologos's propagator network is built on three architectural principles:

1. **Persistent/immutable data structures**: The entire network is a pure, immutable value. Each operation (cell creation, propagator addition, cell write) returns a *new* network, leaving the old one unchanged. This enables $O(1)$ backtracking (keep the old reference) and safe parallel exploration (fork the network value).

2. **Three-domain unification**: Type inference, QTT multiplicity tracking, and session type verification all operate on the same propagator network. Each domain defines its own lattice and propagators; cross-domain *Galois connection* propagators bridge them.

3. **Trait-based extensibility**: Lattice behavior (bottom, join, contradiction detection) is defined via a `Lattice` trait. New domains can be added at the library level by implementing the trait, without modifying the propagator infrastructure.

8.2 Core Data Structures

The network is built on CHAMP (Compressed Hash Array Mapped Prefix-tree) persistent hash maps, giving $O(\log_{32} N)$ lookup, insert, and delete with structural sharing.

```
struct cell-id(n : Nat)          ;; unique cell identifier
struct prop-id(n : Nat)          ;; unique propagator identifier

struct prop-cell
  value      : Lattice           ;; current lattice element (starts at )
  dependents : CHAMP(prop-id)    ;; propagators to fire on change

struct propagator
  inputs  : List(cell-id)        ;; cells this reads from
  outputs : List(cell-id)        ;; cells this may write to
  fire-fn : Network → Network    ;; pure state transformer

struct prop-network
  cells         : CHAMP(cell-id → prop-cell)
  propagators   : CHAMP(prop-id → propagator)
  worklist      : List(prop-id)  ;; propagators waiting to fire
```

```
  fuel          : Nat            ;; step limit to prevent runaway
  contradiction : cell-id | #f   ;; first cell that reached
  merge-fns     : CHAMP(cell-id → (a b → merged))
  ...                            ;; + contradiction-fns, widen-fns,
                                 ;;   decomposition registries
```

8.3 The Three Lattice Domains

8.3.1 Type Lattice

```
type-bot (, no information - fresh metavariable)
    ↓
T (concrete type expression)
    ↓
type-top (, contradiction - incompatible types)
```

The merge function attempts *pure* unification (side-effect free) of two type expressions. If unification succeeds, the result is the unified type. If it fails, the result is $\top$ (type error).

A critical design choice: the type lattice's merge function uses only *read-only callbacks* to resolve metavariables, breaking the circular dependency between the metavar store (which uses the network) and the type lattice (which the network uses).

8.3.2 Multiplicity Lattice

```
mult-bot (, no information)
    ↓
m0 / m1 / mw (erased, linear, unrestricted - mutually incomparable)
    ↓
mult-top (, contradiction)
```

This is a flat lattice: $\bot \sqcup x = x$, $x \sqcup x = x$, $x \sqcup y = \top$ for $x \neq y$. Multiplicities track how many times a value is used, supporting Quantitative Type Theory (QTT).

8.3.3 Session Lattice

```
sess-bot (, no information)
    ↓
Send(A, S) / Recv(A, S) / Choice{...} / Offer{...} / End / ...
    ↓
```

```
sess-top (, protocol violation)
```

Session types describe communication protocols. Merge performs structural unification: same-polarity sessions (send + send) merge component types and continuations; different-polarity sessions (send + recv) produce $\top$ (protocol violation). Choice labels merge covariantly (intersection); offer labels merge contravariantly (union).

8.4 Cross-Domain Propagation via Galois Connections

Different lattice domains are connected by **Galois connection propagators** — pairs of monotone functions (α, γ) that bridge information between domains:

```
;; Type → Multiplicity bridge:
;;    extracts multiplicity info from type structure
;;    returns  (types don't learn from multiplicities)

;; Session → Type bridge:
;;    extracts message type from session step
;;    returns  (sessions don't learn from types)
```

Each Galois connection creates two unidirectional propagators (one per direction). The α direction (concrete $\rightarrow$ abstract) extracts information; the γ direction typically returns $\bot$ (the abstract domain doesn't constrain the concrete domain). More sophisticated connections can propagate information in both directions.

8.5 Scheduling Strategies

8.5.1 Gauss-Seidel (Sequential)

The default scheduler processes one propagator at a time:

```
while worklist  empty:
  pop propagator p from worklist
  apply fire-fn(p) to network, producing network'
  if any output cell changed: enqueue dependent propagators
  decrement fuel
```

This is the simplest and typically fastest strategy for single-threaded execution.

8.5.2 BSP (Parallel)

The Bulk Synchronous Parallel scheduler processes all worklist entries in parallel against a frozen snapshot, then bulk-merges results:

```
while worklist  empty:
  Round k:
    fire ALL propagators in worklist (in parallel, against snapshot)
    bulk-merge all cell changes
    enqueue affected propagators for next round
```

By the CALM theorem, both strategies converge to the same fixed point. The BSP strategy can leverage multiple cores but may require more rounds (Jacobi iteration converges more slowly than Gauss-Seidel for chain topologies).

8.6 Structural Decomposition

When the network unifies two compound types (e.g., $\mathrm{Pi}(A_1, B_1)$ with $\mathrm{Pi}(A_2, B_2)$), the unification propagator creates *sub-cells* for the components (A_1 vs A_2, B_1 vs B_2) and registers new sub-propagators to unify them. A deduplication registry prevents the same compound pair from being decomposed twice.

This on-demand structural decomposition is key to handling recursive and compound types efficiently: cells and propagators are created only as needed, keeping the network sparse.

8.7 Example: Type Inference as Propagation

Consider type-checking the expression `f x` where `f : Int -> Bool` and `x : Int`:

1. Create cells: c_f (type of `f`), c_x (type of `x`), c_r (type of result).
2. Write known types: $c_f \leftarrow \mathrm{Int} \to \mathrm{Bool}$, $c_x \leftarrow \mathrm{Int}$.
3. Add application propagator: "if $c_f = A \to B$ and c_x is compatible with A, then $c_r \leftarrow B$."
4. Propagator fires: c_f is $\mathrm{Int} \to \mathrm{Bool}$, c_x is Int. Merge c_x with domain type $A = \mathrm{Int}$: consistent. Write $c_r \leftarrow \mathrm{Bool}$.
5. Quiescence: all cells stable. Result type is Bool.

If `x` had been `"hello"` (type String), step 4 would merge String with Int, producing $\top$ (type error) — with a dependency chain explaining that the error arose from the application of `f` to `x`.

8.8 Propagators as Universal Substrate

The deepest validation of this architecture is that *the same infrastructure serves all three domains without modification.* The propagator network does not know or care whether a cell holds a type, a multiplicity, or a session protocol. It only knows that cells have merge functions, propagators have fire functions, and the network runs to quiescence.

This means that the three domains compose for free: type inference, multiplicity tracking, and session type verification run simultaneously on the same network, with Galois connections bridging information between them. A type error can surface from a session type constraint; a multiplicity violation can arise from a type structure. The propagator network routes information to wherever it is needed, automatically.

9 Future Directions

9.1 Theorem Proving via Abstract CDCL

D'Silva et al. [14] showed that CDCL generalizes from Booleans to arbitrary lattices. Combined with an ATMS, this yields a propagator-native theorem prover: cells hold lattice values, propagators encode inference rules, ATMS manages hypothetical reasoning and conflict analysis. This could enable type-level theorem proving and automated capability verification without an external SMT solver.

9.2 User-Defined Constraint Domains

Because lattice behavior is defined by a trait, users of a propagator-based language can define their own constraint domains entirely at the library level. A string-length-bound lattice, a nullability lattice, or a permission lattice can be defined, populated with propagators, and connected to the type system via Galois connections — all without modifying the core infrastructure.

9.3 Distributed Propagation

Since propagator cells satisfy CRDT merge requirements, a propagator network can be partitioned across nodes. Boundary cells are replicated and syn-

chronized via lattice merge. Message ordering is irrelevant (commutativity), and convergence is guaranteed (CALM theorem). This enables distributed type checking, multi-agent constraint solving, and decentralized belief propagation.

9.4 Incremental and Live Programming

Persistent propagator networks enable "what-if" exploration: fork the network, make a hypothetical change, observe consequences, and discard the fork at zero cost. Combined with IDE integration, this enables live type checking where edits propagate incrementally to diagnostics, completions, and refactoring suggestions.

9.5 Provenance and Explainability

ATMS dependency chains provide causal explanations for every derived value. Combined with provenance semirings [25], this enables rich answers to questions like "why does this cell have this type?" "what assumptions contributed?" and "what is the minimal change to resolve this error?" — transforming compiler diagnostics from symptoms to explanations.

10 References

[25] Green, T.J., Karvounarakis, G., & Tannen, V. (2007). "Provenance Semirings." *PODS 2007*.